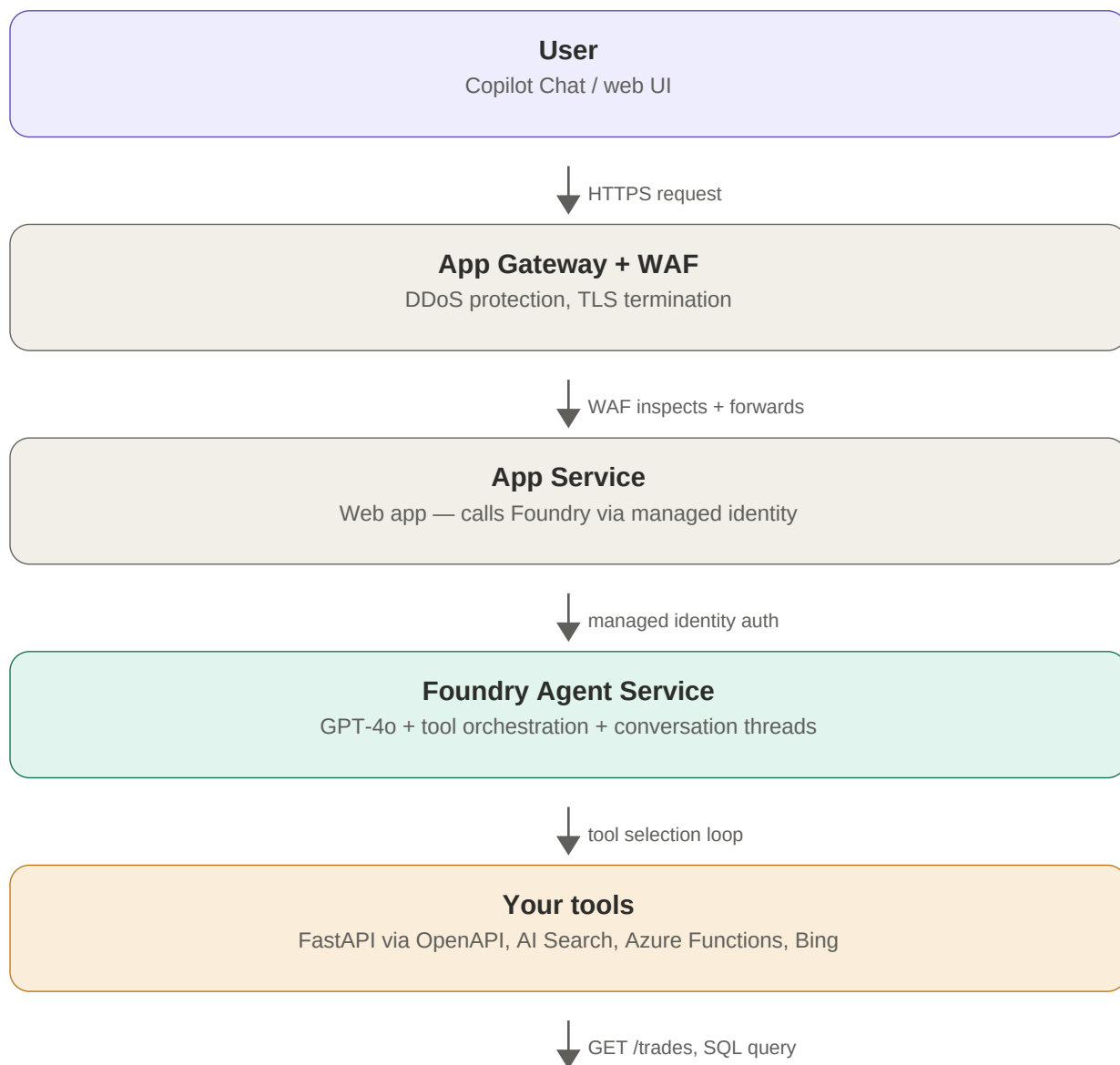


Azure AI Foundry — Agent Architecture

This document explains how Azure AI Foundry works as an enterprise-grade agent platform and how it compares to a Railway-based MCP stack. It covers the full request flow, the tools the agent can use, and how your existing FastAPI trades API plugs in.

1. End-to-end request flow

Every user request travels through five layers inside the Azure private VNet before a response is returned. Managed identities handle authentication at every hop — no API keys in environment variables.



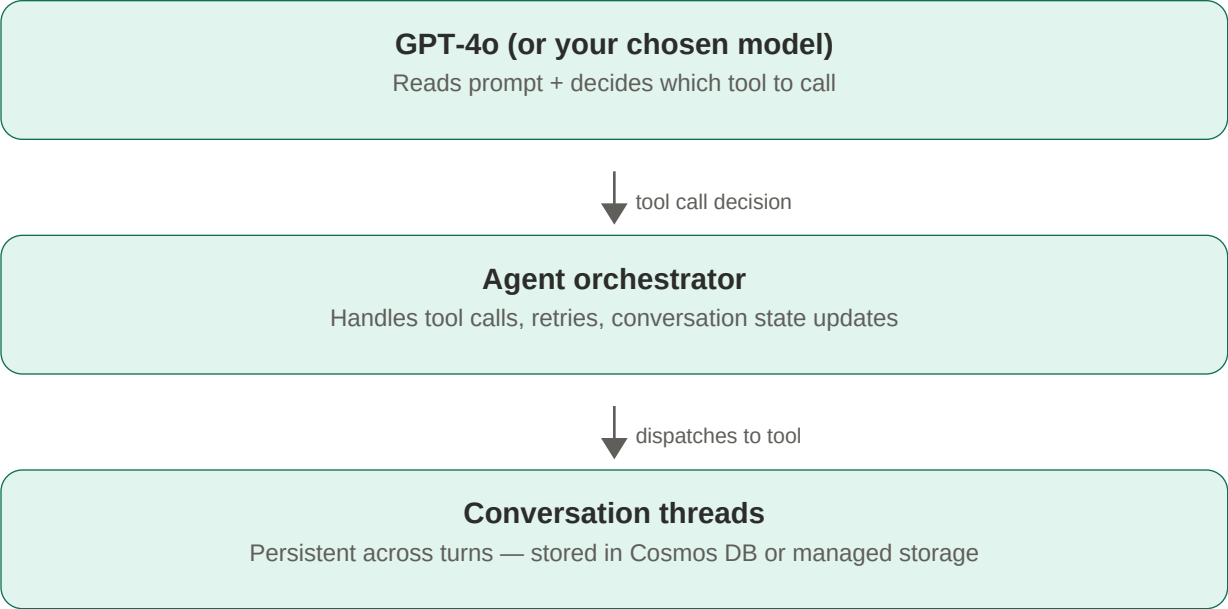
PostgreSQL / data

Trade data returned to agent

The dashed return path carries: SQL rows → FastAPI JSON → tool result → GPT-4o reasoning → formatted answer → user.

2. Inside the Foundry Agent Service

The Foundry Agent Service is fully managed — you don't run any servers for the LLM loop, state management, or tool orchestration. You only provide the tools.



Tools available to the agent

You register tools when creating the agent. Foundry calls them automatically based on the model's decision — no manual routing code needed.

Tool	What it does	Your trades use case
OpenAPI tool	Calls any REST API defined by an OpenAPI spec	Point to your FastAPI /openapi.json — Foundry discovers <code>get_trades</code> , <code>create_order</code>
Azure AI Search	Vector + keyword search over documents	Index trade history, reports, or knowledge base for RAG queries
Azure Functions	Run custom serverless code as agent actions	Trigger alerts, send emails, run batch analysis on trade data
Bing search	Live web data (built-in, no config)	Look up current stock prices or news during trade analysis
Azure Logic Apps	Workflow automation and integrations	Connect to external trading platforms or notification systems

3. Authentication and security

This is the biggest difference from the Railway stack. Every service call in Foundry uses Azure managed identities — no secrets in environment variables, no API keys to rotate.

Layer	Auth mechanism	What it protects
User → App Gateway	Azure AD / Entra ID OAuth 2.0	Who can access the chat UI
App Service → Foundry	System-assigned managed identity	App calls Foundry without credentials
Foundry → your FastAPI	Managed identity + Entra ID OBO flow	Only Foundry can call your API
Foundry → AI Search	Managed identity	Private vector search access
Secrets (DB passwords)	Azure Key Vault reference	Never in env vars or code
RBAC	Azure role assignments	Who can build/run/read agents per project

4. Observability — what you get for free

Foundry captures traces, logs, and evaluations at every step. This is production-grade observability that would take weeks to build on Railway.

Feature	What it gives you
Application Insights	Full request traces from user prompt → tool call → response
Conversation threads	View every turn, every tool call, every intermediate result in the portal
Evaluations	Run automated quality checks on agent responses — groundedness, relevance, coherence
Model router	Dynamically pick the best model per prompt — balances cost, speed, quality
Prompt tracing	See the exact prompt sent to the model at each step — debug hallucinations

5. How your trades FastAPI plugs in

Your existing FastAPI on Railway connects to Foundry as an OpenAPI tool. You point Foundry at your `/openapi.json` and it automatically discovers all your endpoints as callable tools — no MCP server needed.

Step 1	Go to Azure AI Foundry portal (ai.azure.com) → your project → Agents → New agent
Step 2	Select GPT-4o as the model, write a system prompt describing your trades agent
Step 3	Click Add tool → OpenAPI → paste your FastAPI URL: <code>https://trades-api-production.up.railway.app/openapi.json</code>
Step 4	Foundry reads the spec and discovers: GET <code>/trades</code> , GET <code>/trades/{id}</code> , POST <code>/trades</code>
Step 5	Add auth header — create a Key Vault secret for your API key, reference it in the tool config
Step 6	Click Try in Playground and type: 'Show me all BUY trades for AAPL'
Step 7	Deploy the agent — expose it via App Service or integrate with Copilot Extension

6. Railway stack vs Azure AI Foundry

Both approaches produce a working agent. The right choice depends on your scale, compliance requirements, and whether you need enterprise integrations.

	Railway + MCP (what you built)	Azure AI Foundry
Cost	\$5/month	Pay-per-token + Azure compute
Setup time	Hours	Days (enterprise config)
Auth	Env vars / API keys	Entra ID + managed identities
LLM control	Full (you pick model)	Full (GPT-4o, o1, custom)
State mgmt	You build it	Managed threads (Cosmos DB)
Observability	You add it (Sentry)	Built-in (App Insights + evals)
VNet / private	No	Yes — full private endpoint support
Compliance	None built-in	SOC 2, ISO 27001, HIPAA eligible
Copilot integration	MCP or Skillset extension	Copilot Extension (same path)
Best for	Side projects, prototypes	Enterprise, regulated industries

Recommendation: keep Railway for development and prototyping. Migrate to Azure AI Foundry when you need Entra ID auth, compliance logging, private networking, or enterprise Copilot distribution.